Prayog — Bug / Defect History and Fixed-Defect Evidence

CS455 Deliverable 3 · Group 3 · 6 November 2026

1. Summary

	Count
Defects recorded for D3 (D3-01 … D3-20)	20
Fixed, each with a regression test	**19** (all except D3-12, a CI fix verified by a green run)
Open	0
Found by: requirements re-check 1 · code review 4 · CI 1 · concurrency testing 1 · load and benchmark testing 4 · fault injection 2 · static analysis 1 · adversarial and security testing 6	
Earlier defect records	18 architecture defects (AD-01…18, D1 verification package), the D2 log's CI test defect, the Steward log's 14 AI errors

Jira. The 16 October Jira export contains no issues of type *Bug*: defects were fixed inside story branches and reviews rather than ticketed. To give the Jira history the deliverable asks for, every defect below is in `evidence/jira_bugs_import.csv` (Issue Type = Bug, priority, epic link, status, fixing commit), ready for *Jira → System → External system import → CSV*. Once imported, the bugs appear in the dashboard's created-vs-resolved and open-bug gadgets (Project Report section 4).

2. Meaningful defects found and fixed in D3 testing (6 November)

Each was found by a test technique of this deliverable, not by the functional suites (455 of which passed on the same code). For each: the evidence before, the change, and a regression test that fails without the change.

D3-03 — One inconsistent GPU unit row stopped all scheduling (High)

Found by	concurrency suite, test C3 on the shared test database
Symptom	`IntegrityError: duplicate key value violates unique constraint "ux_alloc_units_live"` escaping `run_pass` — every later pass died at the same claim
Root cause	the claim treated the partial-unique-index backstop firing as fatal
Fix	`logic/gpu/scheduler/claim.py`: roll back, log ERROR naming job and node, return `LOST_CLAIM_RACE`; the pass continues
Regression test	`test_concurrency.py::test_c6_one_inconsistent_unit_does_not_stop_scheduling` — **fails on `main`, passes after**
Report	Concurrency report section 4

D3-04 — `GET /jobs` was N+1: 674 ms for 100 jobs (Medium)

Found by	code review (2 Nov, left open), quantified by the load test
Symptom	674 ms median for one user; throughput flat at 1.9 req/s
Root cause	`job_status` per job: 3–6 queries each + a full node reload per queued job
Fix	`job_statuses()` batch in `logic/gpu/scheduler/requests.py`; `job_status` shares the same code
Regression tests	`test_job_list_batched.py` (3): field-by-field equality with the single-job view for every state; query count constant from 8 to 39 jobs — import fails on `main` (function absent)
Result	37 ms median (18×), 34.7 req/s

D3-05 — Run output directory reachable by any local account on a GPU node (Medium, security)

Found by	bandit B103 (High) triage: `os.chmod(dirs["output"], 0o777)`
Root cause	`output/` must be world-writable for uid 65534 in the container, but its parent workspace kept the default umask, so others could traverse into it and plant or replace files before upload
Fix	`node_agent/workspace.py`: workspace `chmod 0o700` (the container mounts `output/` directly, so it is unaffected)
Regression test	`test_node_agent_units.py::test_good_code_is_joined_verified_and_extracted_read_only` asserts `0o700` on POSIX (runs in CI on Linux; skipped on Windows, which has no such bits)

D3-06 — Database outage answered with a bare 500 (Medium, reliability)

Found by	failure drill F3 (PostgreSQL stopped): `500, 500, 500`
Root cause	no handler for `OperationalError` / pool timeout
Fix	`main.py`: 503 + `Retry-After: 5`, generic message, driver detail only in the server log
Regression tests	`test_database_unavailable.py` (2) — fail on `main` (500), pass after; drill F3 re-run (Failure report section 4)

D3-07 — Connection-pool exhaustion: 30 s hangs and 31% errors at 64 users (High, performance)

Found by	load test: `QueuePool limit of size 5 overflow 10 reached` ×32 in the server log
Root cause	40 request threads, 15 pooled connections, 30 s pool timeout
Fix	`database.py`: `PRAYOG_DB_POOL_SIZE` 20, `PRAYOG_DB_MAX_OVERFLOW` 20, `PRAYOG_DB_POOL_TIMEOUT_SECONDS` 10
Evidence	`GET /jobs/{id}` at 64 users: 2.6 → 60.6 req/s, errors 31% → 0%; `POST /jobs` at 8 users: errors 23% → 0% (Performance report section 4.2)

D3-08 — An API or database outage was mistaken for node failure (High, reliability)

Found by	failure drills F2 and F3 (shortened timeouts), reproduced at production timings by `backend/experiments/failure_experiment.py`
Symptom	API stopped for 45 s with the tick running: the healthy node went `DOWN`, its running job was requeued, a false `NODE_DOWN` alert was raised

Root cause	the reaper compared `last_heartbeat_at` with the clock and could not tell "the node stopped" from "the platform stopped listening"
Fix	`logic/gpu/scheduler/liveness.py`, table `platform_liveness`, migration `a9c1e7d3f5b2`: each API process stamps a row every 2 s. The reaper judges heartbeats and dispatch timeouts only when the stamp is fresh and the API has been up for a full heartbeat timeout since its last gap
Regression tests	`test_cases/gpu/scheduler/test_liveness.py` (9): the outage cases, a node that really died is still detected, an outage uses up no dispatch attempt
Evidence	same scenario on the fixed build: node stays `AVAILABLE`, job stays `RUNNING` on the same allocation, 0 alerts; a real node death is still detected in 33.9 s (Failure report section 5)

D3-14 — Overlapping passes cost jobs × nodes × passes (High, performance)

Found by	`backend/experiments/pass_benchmark.py`, four concurrent passes at growing queue sizes
Symptom	one pass at 400 jobs took under a second; four at once took 51 s each
Root cause	a pass that lost a race kept working from its stale snapshot: every remaining job tried every node it believed free, each a full locking transaction followed by a rollback
Fix	`_Pass._learn` in `logic/gpu/scheduler/passes.py`: a failed claim updates this pass's view of the node. The claim transaction still decides correctness
Regression tests	`test_adversarial.py::test_a_lost_race_is_not_retried_for_every_job_on_every_node` (first failed with 90 claims instead of 3), `test_a_node_that_left_available_is_not_retried`, `test_a_lost_race_for_two_gpus_still_lets_a_one_gpu_job_try`; `test_pass_equivalence` still passes

D3-15 — Every event started its own pass, so a burst collapsed the API (High, performance)

Found by	`backend/experiments/load_test.py` with node agents running: at 50 users, 8.6 req/s and a submit p95 of 8.6 s
Root cause	each submit, cancel, edit and run end started a full pass; passes stacked up and fought over the same rows and connections
Fix	`trigger_pass` in `logic/gpu/scheduler/passes.py`: at most one running and one owed per API process. The owed pass starts after the caller committed, so it sees the change
Regression tests	`test_a_burst_of_triggers_runs_one_pass_plus_one_owed`, `test_a_failed_pass_does_not_stop_later_triggers`

D3-16 to D3-20 — Found by adversarial and security tests through the real API

ID	Defect	Severity	Before	Fix	Regression test (`test_cases/gpu/scheduler/`)
D3-16	A NUL character in any stored string crashed the request	Medium	`500` on `POST /jobs` (3 fields) and `POST /admin/nodes`	`main.py` answers 400 `INVALID_VALUE` for every route	`test_a_nul_character_in_a_job_field_is_refused_not_a_server_error`, `test_a_nul_character_in_a_node_name_is_refused`
D3-17	Out-of-range numbers from a node agent crashed the request	Medium	`500` for `seq`, `exit_code` and `unit_count` beyond the column range	bounds in `schemas/gpu/agent.py` (422), and `DataError` answers 400	`test_an_absurd_unit_count_in_a_heartbeat_is_refused`, `test_an_out_of_range_agent_report_is_refused_and_the_job_keeps_running`
D3-18	**Authorization:** naming someone else's snapshot on `POST /jobs` allowed `POST /rebuilds` on it	High	`201` and a rebuild attempt on the victim's snapshot; the same route also passed an owner's INCOMPLETE snapshot	`logic/mlflow/rebuild.py`: a snapshot the ledger holds is rebuildable only through the ledger's own check	`test_naming_someone_elses_snapshot_on_a_job_does_not_let_you_rebuild_it`
D3-19	A correctly signed token with no expiry was accepted	Low	`200`	`logic/auth/dependencies.py` requires `exp` and `sub`	`test_a_signed_token_without_an_expiry_is_refused`
D3-20	A refused cancel left no audit row, though scenario A-18 requires one	Medium	application log only	`logic/gpu/scheduler/cancel.py` writes `job.cancel` / `REFUSED`	`test_a_refused_cancel_leaves_an_audit_record`

The failing runs before these fixes are kept in `evidence/adversarial_before_fix.txt`, and the passing runs in `evidence/adversarial_after_fix.txt`.

3. Full defect register

ID	Defect	Severity	Found	How	Fix	Regression evidence	Status
D3-01	CLI rebuilds created no `RebuildAttempt`; failed CLI rebuilds never reached S3, successful ones never S4 (FR-STW-001/006, FR-INT-01)	High	29 Oct	requirements re-check against `main`	`8a38f79`	`test_cli_rebuild_reaches_steward.py` (2) — `637bf75`	fixed, merged (PR #26)
D3-02	No failed-sign-in lockout (FR-AUTH-60): unlimited guessing	Highest	2 Nov	full code review	`717f1fc` + migration `b7d4e2a1c9f0`	`test_review_fixes` (4 lockout tests), concurrency C4, probe A19	fixed, merged
D3-03	One inconsistent unit stopped all scheduling	High	6 Nov	concurrency testing	`claim.py`	concurrency C6	fixed (`f358e01`)
D3-04	`GET /jobs` N+1	Medium	2 Nov / 6 Nov	review + load test	`requests.py`, `jobs.py`	`test_job_list_batched` (3)	fixed (`f358e01`)
D3-05	World-reachable run output on GPU nodes	Medium	6 Nov	static analysis	`workspace.py`	`test_node_agent_units` (POSIX)	fixed (`f358e01`)
D3-06	DB outage → 500	Medium	6 Nov	fault injection	`main.py`	`test_database_unavailable` (2), drill F3	fixed (`f358e01`)
D3-07	Pool exhaustion under load	High	6 Nov	load testing	`database.py`	load re-run	fixed (`f358e01`)

ID	Defect	Severity	Found	How	Fix	Regression evidence	Status
D3-08	API/DB outage > heartbeat timeout marks healthy nodes DOWN and requeues running jobs	High	6 Nov	fault injection (F2, F3)	`liveness.py`, `reaper.py`, migration `a9c1e7d3f5b2`	`test_liveness` (9), failure experiment	fixed
D3-09	Handover stranded runs in flight (uploads looked up under the old owner)	High	2 Nov	code review	`717f1fc`	`test_review_fixes::test_a_handover_leaves_a_placed_run_able_to_start_and_store_its_results`	fixed, merged
D3-10	Approved Steward fix queued a job with no code	High	2 Nov	code review	`717f1fc`	`test_review_fixes::test_an_approved_steward_fix_runs_the_same_code_again`	fixed, merged
D3-11	Project create / upload / queue did not need active access	High	2 Nov	code review	`717f1fc`	`test_review_fixes` (2 access tests)	fixed, merged
D3-12	CI red: no Python 3.9 on `ubuntu-latest` (26.04); e2e user lacked access after D3-11	Medium	4 Nov	CI	`cc209d9`	CI green on PR #26	fixed, merged
D3-13	`POST /jobs/{id}/cancel` answers 403 for another user's job, 404 for a missing one: job IDs can be enumerated	Medium	6 Nov	adversarial testing (A03b)	`cancel.py`: 404 for both, the refusal audited (D3-20). FR-GPU-065's "insufficient permission" wording yields to NFR-GPU-030	`test_another_users_job_cannot_be_read_edited_or_cancelled`, probe A03b	fixed
D3-14	Overlapping passes cost jobs × nodes × passes	High	6 Nov	benchmark	`passes.py _learn`	`test_adversarial` (3)	fixed
D3-15	Every event started its own pass; a burst collapsed the API	High	6 Nov	load testing	`passes.py trigger_pass`	`test_adversarial` (2), load re-run	fixed
D3-16	NUL character crashed the request	Medium	6 Nov	adversarial testing	`main.py`	`test_adversarial` (4)	fixed
D3-17	Out-of-range agent values crashed the request	Medium	6 Nov	adversarial testing	`schemas/gpu/agent.py`, `main.py`	`test_adversarial` (4)	fixed
D3-18	Someone else's snapshot could be claimed and rebuilt	High	6 Nov	security testing	`logic/mlflow/rebuild.py`	`test_adversarial` (1)	fixed
D3-19	Token with no expiry accepted	Low	6 Nov	security testing	`logic/auth/dependencies.py`	`test_adversarial` (1)	fixed
D3-20	Refused cancel left no audit row	Medium	6 Nov	security testing	`cancel.py`	`test_security` (1)	fixed

4. Earlier defect history (Deliverables 1 and 2)

Period	Records	Where
D1 verification (Sep)	18 architecture defects AD-01…AD-18 against SADD v1.0 plus cross-document conflicts; resolved in the Integrated Requirements Specification and SADD v2.0	`final deliverables/04_Consolidated_Defect_Register.md`
D2 ML Flow (Oct 12–19)	CI exposed a test that inspected the wrong `db.add` call; repeat-sync metadata instability (`cb0a10b`); e2e project-name collisions (`1975569`); rebuild e2e script newlines (`772899f`)	repository `AI_ENGINEERING_LOG.md`, git history
D2 Steward (Oct 5–15)	14 defects in AI-written code caught by tests, live runs and review (e.g. `rowcount` −1 on a guarded update, double model load on cold start, UTC offsets) — and the unreviewed push to `main`, reverted in `d11d488`	`Steward_AI_Engineering_Log.md`
GPU scheduler build (13–15 Oct)	Found before merge by mutation testing (51 deliberate bugs, 4 tests added because a bug survived): a pass could write a queue reason onto a job another pass had just placed; the free-unit metric ignored maintenance; a race between an agent confirming a start and a dispatch timeout could requeue a job that had just started	Fixed before review; regression tests in `test_cases/gpu/scheduler/`
29 Oct – 5 Nov review	27 findings: 21 fixed (`717f1fc`), 6 left open (staged-object delete race, self-declared rebuilds via PATCH, no email verification, checkpoints > 250 MiB skipped silently, cancel's 403, `GET /jobs` per-job queries → now D3-04)	section 3 above

5. What the defect data says

- **Where defects were found.** Of the 20, only one came from the functional suites' normal route (CI); five came from code review or a requirements re-check, and fourteen from this deliverable's non-functional testing. The functional suites were green on code that still had D3-03 to D3-08 and D3-13 to D3-20.
- **Severity.** 11 of 20 are High or Highest. None is open.
- **The two most serious late finds were invisible to every earlier test.** D3-08 needs a real outage at production timings, and D3-14 needs several passes overlapping on a long queue.
- **Process change for Sprint 3.** Log defects as Jira Bugs when found, not only in commits. The experiments now live in `backend/experiments/`, and CI enforces 80% coverage.